CS 440 – Operating Systems
Project 4 – Cache Replacement Algorithms
50 Points Available
Due in Canvas by 11:59 PM on Tuesday, April 21, 2026

Instructions

In this project, you will implement and compare the behavior of six cache replacement algorithms:

1. First-In First-Out (FIFO)
2. Least Recently Used (LRU)
3. Least Frequently Used (LFU)
4. Optimal (MIN)
5. Most Recently Used (MRU)
6. Random (RAND)

Refer to the Chapter 10 supplemental slides for formal definitions and behavior.

User Inputs

Your program must prompt the user for the following:

1. Page reference pattern length

- Length of the generated reference string
- Range: 10 to 100 (inclusive)

2. Number of unique pages

- Number of distinct pages (A, B, C, …)
- Range: 2 to 15 (inclusive)

3. Number of cache slots (frames)

- Number of slots available in memory
- Range: 2 to 10 (inclusive)

4. Random number seed

Program Requirements

Reference String Generation

- Use uppercase letters starting from **A**
- Example: If there are 5 pages → **A, B, C, D, E**
- Generate the reference string of pages using these letters randomly
- Each page must have **equal probability**

Simulation Rules (All Algorithms)

1. **Initial Fill Phase**
    - Fill slots sequentially (slot 1 → slot N)
    - No replacement occurs until all slots are filled

2. **Cache Hit**
    - If a page is already in cache → mark with **"+"**
    - No replacement occurs
3. **Cache Miss**
    - Apply the algorithm's replacement policy

Special Rules

- **RAND**
    - After cache is full, randomly select a slot for replacement
- **MIN (Optimal)**
    - Replace the page that will be used **farthest in the future**
    - If future use cannot be determined (end of string), replace:
      → **lowest-numbered slot**
- **LFU (Least Frequently Used)** algorithm:
    - Each page must maintain a **reference count** (number of times it has been accessed while in cache).
    - On a cache miss, replace the page with the **lowest frequency**.

  Tie-breaking rule (REQUIRED):

  If multiple pages have the same lowest frequency:

    - Select the page that was **least recently used (LRU among the tied pages)**
    - If still tied (rare edge case), replace the page in the **lowest-numbered slot**

Output Requirements

- Display the reference string clearly (space-separated)
- Align all algorithms vertically with the reference string
- Show:
    - Page placements
    - Cache hits using **"+"**
- Ensure readability for long sequences (handle wrapping if needed)

Your program will prompt the user for these items, then correctly map and calculate the number of page hits and page misses for each algorithm following the algorithms studied in class. Show the plus sign (+) when there is a *cache hit*, meaning the page was already in memory and no replacement was necessary. See the example below (the red box indicates the page reference pattern length; the blue box indicates the number of slots for each page replacement scheme). Note: you do not have to put the boxes; they are simply for illustration.

```
Enter page reference pattern length: 12

Enter number of unique pages: 5

Enter number of slots: 3

Enter a seed:  5418
```

```
Ref Str: A B C D A B E A B C D E
 FIFO 1: A A A D D D E E E C C C
 FIFO 2:   B B B A A A A B B D D
 FIFO 3:     C C C B B B B A A E
        - - - - + + - + + - - -

  LRU 1: A A A D D D E E E C C C
  LRU 2:   B B B A A A A B B D D
  LRU 3:     C C C B B B B A A E
        - - - - + + - + + - - -

  LFU 1: A A A D D D E E E C C C
  LFU 2:   B B B A A A A B B D D
  LFU 3:     C C C B B B B A A E
        - - - - + + - + + - - -

  MIN 1: A A A D D D D D D D D D
  MIN 2:   B B B B B B B B B B B
  MIN 3:     C C A A E E E C C E
        - - - - + + - + + + + +

  MRU 1: A A A D D D E E E C C C
  MRU 2:   B B B A A A A B B D D
  MRU 3:     C C C B B B B A A E
        - - - - + + - + + - - -

 RAND 1: A A A D D D E E E C C C
 RAND 2:   B B B A A A A B B D D
 RAND 3:     C C C B B B B A A E
        - - - - + + - + + - - -

Cache Hit Rates:
FIFO : 4 / 12 = 0.33
LRU  : 4 / 12 = 0.33
LFU  : 4 / 12 = 0.33
MRU  : 4 / 12 = 0.33
RAND : 4 / 12 = 0.33
MIN  : 6 / 12 = 0.50

Best Algorithm: MIN
Worst Algorithm(s): FIFO, LRU, LFU, MRU, RAND (tie)
```

We see that <**Number of unique pages**> pages A, B, C, D, and E, are randomly created as a string of length <**Page reference pattern length>** and fit into **<Number of slots>** slots for each algorithm.

From the number of unique pages, determine the letter order to be shown in the reference string using a random number generator. Use capital letters (e.g., A, B…), not numbers, for the reference pages. Begin at 'A' and end at the letter representing the number of unique pages (i.e., 'H' for 8). *Randomly* determine the reference string using the reference pages. Each page (i.e. letter) should have an equal probability.

For all algorithms, fill all available slots (in slot number order) first - see the first three pages loading A, B, and C in the example shown earlier. Next, determine the slot for each page (if not already in the cache) using the algorithm.

For the **Random** algorithm, fill all available slots (in slot number order) first. After all slots are filled, check for any cache hits. If there is no cache hit, randomly determine which slots will be replaced.

For the **MIN** algorithm, if at the end of the reference string, and you cannot determine which slot to replace due to limited lookahead information (and no cache hit), replace the slot with the *lowest* slot number.

Focus on *readability*. In your display, make sure you put at least 1 blank space between each page (letter) in the reference string. Align each algorithm's page with the reference string as shown above. Ensure it works when the display is beyond the normal screen display (think of how to wrap the information for large pattern lengths)

Provide *statistics* on the hit rates (see example above for what to provide), as well as provide a best (or set of best) algorithm(s) and worst (or set of worst) algorithm(s). There is the possibility of earning extra credit for providing innovative, useful statistics beyond the project requirements above.

Provide *screenshots* from several runs (between 3 and 5). Annotate the runs where appropriate.

Write up a *paragraph* or two about what you observed from your runs. Did each perform better/worse/same as you expected? Why or why not?

General Guidelines

- This project can be done as an individual assignment or as a team of two. If you work in pairs, ensure you include who you are working with in the comments of your program and in Canvas upon submission.
- Submit all work through the dropbox in Canvas. Include your code as well (don't just provide a GitHub link; you can do this as a backup if you wish).
- Create a screen-cast-o-matic (or another video narration tool) demonstrating your code and runs/outputs; a deduction of 5 points will be made if you do not provide this.
- You can use any language you wish for this, but for languages other than Python/Java/C++/C#/C/Rust/Go, please talk with me first.

Grading

- 30 points: code working as described above
- 5 points: screenshots provided of *at least* 3 runs, one of which is wider than the screen width
- 5 points: output format is as in the example above
- 10 points: write-up of findings